Transaction Integrity Eval

Experiment report: design, grader validation, model trials, results and conclusions
micro1 Eval Hackathon 2026 · El-Moatasem Madani · Runs recorded 8 November 2026 · Repository: github.com/El-Moatasem/transaction-integrity-eval

147	145 / 147	84 / 84	0 / 147
recorded agentic trials	passed (98.6%)	trials with race tests: all race tests passed	transcripts flagged by network audit

Summary. The eval tests whether AI coding agents preserve payment and ledger invariants when requests are retried, run on several workers at once, or fail halfway through. Three models (Claude Opus 5.5, Sonnet 5.5, Haiku 5.5) were run through 14 Python/SQLite tasks with withheld behavioral graders. Concurrency was never the problem: every race test passed, including when the concurrency requirement was only implied. The two failures, both on the multi-write-path task T14, were **transaction-lifecycle bugs on rare exits**: an idempotent replay that returned with its write transaction still open, and a rollback helper that committed a partial write. The models reliably knew *to* lock; what they occasionally got wrong was closing the transaction on every exit path.

1. Domain and motivation

Money-moving backends (captures, refunds, ledgers, payouts, webhooks) fail in ways that rarely appear on the happy path: a retried request charges twice, two concurrent refunds both pass a ceiling check, an error halfway through leaves a balance changed with no receipt. These bugs are silent, expensive and hard to reproduce, which is exactly where AI-generated code is risky. The eval is aimed at engineers and reviewers who rely on coding agents for transactional code, and at researchers who need graders that can actually detect these failures.

2. Benchmark design

Each task gives the agent an `instruction.md` and a deliberately flawed starter `service.py`. Reference solutions, wrong-fix mutants and the pytest grader are withheld. A trial passes (reward 1) only if every behavioral assertion passes; there is no LLM judge.

ID	Task	Invariant under test
T01	Idempotent payment capture	Same key never debits twice; conflicting reuse changes nothing
T02	Refund ceiling	Committed refunds never exceed the capture
T03	Double-entry transfer	Balanced, exact-cent postings; atomic writes
T04	Webhook ordering	Only legal transitions consume events; terminal states stay terminal
T05	Settlement reconciliation	Duplicate, missing, unexpected and mismatched rows distinguished
T06	FX rounding	Exact decimal conversion, target minor units, half-even
T07	Subscription proration	UTC elapsed seconds, one final rounding
T08	Marketplace splits	Every cent conserved; deterministic remainders
T09	Dispute deadlines	Inclusive UTC deadline; terminal states cannot reopen
T10	Inbox/outbox atomicity	Balance, inbox and outbox commit together; idempotent replay
T11–T13	Implicit-spec ablations of T01, T02, T10	Same graders; all mention of races/transactions/locking removed; only "runs on several worker processes, one SQLite connection per call"
T14	Refund write paths	Bug report about over-refunds; `refund`, `adjust_capture` and `void_remaining` must all preserve the ceiling and the void rule

3. Grader design and validation

3.1 Controls on every task

Each grader must accept the reference solution and reject both the starter code and a deliberately wrong fix. Independent correct alternatives (for example an atomic conditional-UPDATE refund and a deferred-`BEGIN` variant) must also be accepted, so the grader checks persisted outcomes rather than a preferred SQL statement.

3.2 The race probe, and two loopholes found before any model trial

Concurrency tests pause connection A at a chosen point while connection B commits, then release A and check persisted state. The first version paused by wrapping `conn.execute` and matching SQL text. Testing it with broken variants of our own reference solutions exposed two false passes:

- **Cursor-style code skipped the race.** With the transaction removed and SQL issued through `conn.cursor().execute()`, the pause never fired and the grader passed broken T01, T02 and T10 code. On T02 that code commits 1,400 cents of refunds against a 1,000-cent capture.
- **T10 paused too early** (after the inbox read, before the balance read), so a no-transaction T10 passed 5 of 5 runs.

Fix: pause A immediately before its first write using `sqlite3.Connection.set_trace_callback`, which sees every statement regardless of API or SQL wording. A further correction: replacing `BEGIN IMMEDIATE` with a plain deferred `BEGIN` is *correct* on SQLite (one side gets "database is locked" and rolls back, which the contract permits), so it is a positive control, not a mutant. The genuinely broken variant is no transaction at all.

3.3 Final mutation audit

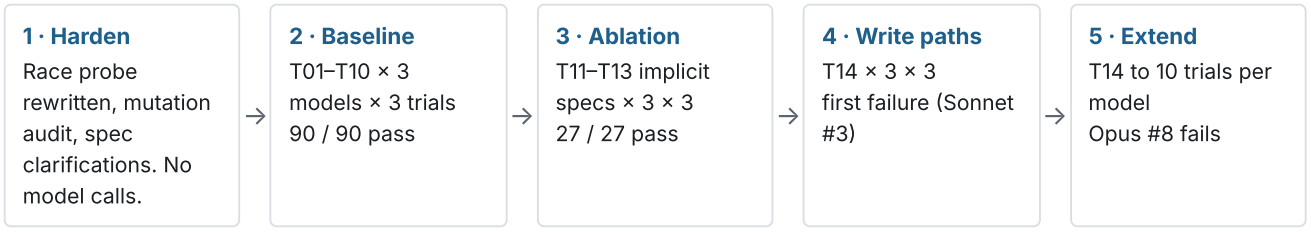
Control class	Result
Valid implementations accepted (references, plain-BEGIN variants, conditional-UPDATE alternative; T01, T02, T10–T14)	16 / 16
Broken concurrency variants detected (no transaction, with and without cursor; T01, T02, T10–T13), race confirmed exercised each time	12 / 12
Must-reject variants rejected (T03 partial journal, T10/T13 partial outbox, T14 refund-only fix)	4 / 4

Repeatability runs confirmed every expected control outcome across repeated rounds. Before the first model trial, four spec clarifications were made so that every graded behavior is stated in the instruction (T01 concurrent replay never raises `IntegrityError`; database errors propagate rather than becoming `ValueError`; T02 nonempty identifiers; T04 boolean sequence invalid). No test assertion or reference solution changed.

4. Experimental setup

Item	Setting
Agent	Claude Code CLI 2.1.319, headless (`claude -p`), stream-json transcripts
Models	`claude-opus-5-5`, `claude-sonnet-5-5`, `claude-haiku-5-5` (model ID confirmed from each transcript)
Prompt (identical for every trial)	"Your task is described in INSTRUCTION.md in the current directory. Read it, then implement the required changes in service.py. Work only inside the current directory. When you are finished, stop."
Tools / limits	Read, Edit, Write, Bash; WebFetch and WebSearch disabled; no MCP servers; max 40 turns; 900 s timeout; temperature at CLI default (not configurable)
Isolation	Fresh temporary workspace outside the repository per trial containing only the instruction and starter code. No network sandbox: a post-run transcript audit scans every tool call for network or answer-file indicators.
Classification	Auth errors, rate limits, crashes, timeouts and missing results are infrastructure failures and are never recorded as model failures. Turn-budget exhaustion is graded on the final code.
Trials	3 per model per task; T14 extended to 10 per model (see chronology)

4.1 Experiment flow



UTC times from the trial records: baseline 16:57–17:51; ablation 17:52–17:59; T14 trials 1–3 18:37–18:42 (first failure 18:41); T14 trials 4–10 18:46–18:56. T11–T13 and T14 were designed after earlier results were seen; this is disclosed, and every trial is reported. The T14 instruction and grader were identical for all 30 T14 trials.

5. Results

Model	T01–T10 explicit	T11–T13 implicit	T14 write paths	Total	Runtime	CLI-reported cost
`claude-opus-5-5`	30/30	9/9	9/10	48/49	2,509 s	$9.77
`claude-sonnet-5-5`	30/30	9/9	9/10	48/49	785 s	$3.15
`claude-haiku-5-5`	30/30	9/9	10/10	49/49	1,241 s	$0.32
Total	**90/90**	**27/27**	**28/30**	**145/147**	**4,535 s**	**$13.24**

- On 13 of 14 tasks, all three models passed every trial (pass^k = 1).
- Race tests exist in T01, T02 and T10–T14: 84 trials, 174 race-test executions, **all passed**. Both failures came from sequential tests.
- Removing all concurrency language (T11–T13) did not reduce pass rates; models inferred atomicity from the deployment fact alone.
- Costs are CLI-reported (API-equivalent) and were not checked against billing.

6. Failure analysis

Both failing solutions placed `BEGIN IMMEDIATE` on all three T14 write paths and passed all four T14 race tests. Both file hashes match the trial records; both failures map to explicit sentences in the instruction.

6.1 Sonnet 5.5, T14 trial 3: leaked transaction on idempotent replay

```
conn.execute('BEGIN IMMEDIATE')
prior = conn.execute('SELECT capture_id,amount_cents,receipt_id FROM refunds WHERE refund_key=?',
...).fetchone()
if prior:
    if prior[0] != capture_id or prior[1] != amount_cents: raise ValueError('idempotency conflict')
    return {...}          # returns with the write transaction still open
...
except BaseException:
    conn.rollback(); raise    # only runs on errors
```

The replay returns the right receipt, but the connection keeps an open write transaction. The next call on it fails with `sqlite3.OperationalError: cannot start a transaction within a transaction` instead of the specified `ValueError` (failing `test_refund_ceiling_replay_and_conflicts` and `test_void_remaining_rules`). In production the leaked transaction holds SQLite's write lock on a pooled connection, so one retried request can stall every other worker.

6.2 Opus 5.5, T14 trial 8: rollback helper that commits

```
def _begin(conn):
    if conn.in_transaction:
        conn.commit()          # "clean up" any open transaction
    ...
def _rollback(conn):
    _begin_safe(conn)          # calls _begin() first ...
    ...
    conn.rollback()            # ... so this rolls back an empty transaction
```

When an injected failure hit the refund-receipt `INSERT`, the error path called `_rollback`, which first called `_begin`, which committed the balance update already made. The final state was `credited: 200` against an expected `0`: money credited with no refund receipt (failing `test_database_errors_propagate_and_roll_back`). This violates "a call that raises leaves the database unchanged" and "no credit exists without its receipt". The bug lives in the more defensive code: an extra safety wrapper turned the rollback into a commit.

7. Conclusions

Key finding. On these tasks, current Claude models reliably serialize concurrent financial writes, even when concurrency is only implied by deployment context and when the bug report points at only one of several write paths. The residual risk is not *whether* they lock but *how they leave* the transaction: early returns and error handlers.

Hot take. Race tests alone would have scored these models perfectly. Graders and code reviews for AI-written transaction code should exercise every exit from the transaction (replay, conflict, injected failure, and the call that follows), not just the concurrent path. A naively built race grader is also a risk in its own right: our first version passed broken code in four cases.

8. Limitations

- **Small sample.** Two failures in 147 trials; the 9/10 versus 10/10 differences on T14 are not a model ranking. The finding is a hypothesis to test at larger scale.
- **Bounded scheduling.** Each race test exercises one chosen interleaving; an implementation that writes, reads, then writes again may not be paused at its decisive read.
- **Isolation.** No network or filesystem sandbox; Bash could reach the network. The transcript audit is a screen, not proof. Trials ran without `--bare`, so user-level Claude Code settings could load.
- **Post-hoc extensions.** T11–T13 and T14 were designed after earlier results, and T14 was extended after its first failure. All trials are reported; none were excluded.
- **One model family and one agent.** Claude models evaluated through Claude Code; AI assistance from the same family helped build the benchmark, a possible source of bias. Harbor/Docker execution was not run.
- **SQLite.** Chosen for offline reproducibility; locking semantics differ from PostgreSQL, and no engine equivalence is claimed.

9. Reproducibility and provenance

- `python run_eval.py validate` and `python scripts/audit_variants.py` reproduce the grader controls offline, with no API key.
- `python scripts/run_claude_trials.py --models <ids> --tasks <ids> --trials N` reruns the trials (requires the Claude Code CLI). Interrupted batches resume without overwriting evidence.
- Evidence: `results/trials.jsonl` (per-trial model, settings, runtime, cost, transcript/code/grader SHA-256), `results/summary.json`, `results/grades/`, `results/final_code/`, `transcripts/` (raw stream-json), `results/network_audit.json`, `results/mutation_audit.json`.
- **AI use.** Claude (Claude Code and chat) was used substantially to build tasks, graders, the race probe, the trial runner and drafts of the documentation, including this report. The author directed the design decisions, approved every task instruction, ran all trial batches, and inspected both failing implementations.